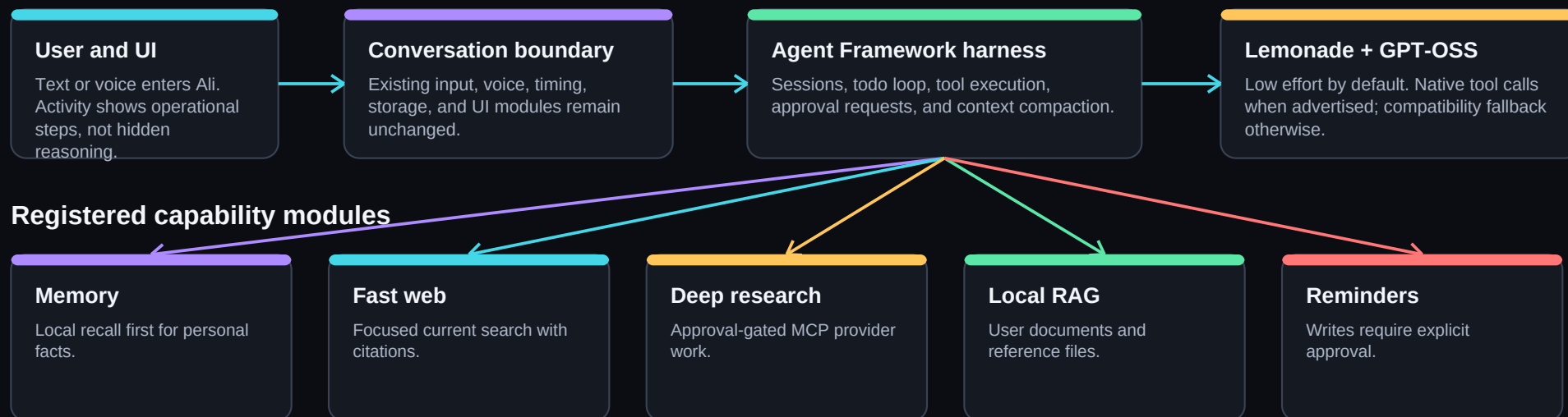


How the pieces fit together

The model receives the whole request and chooses direct response, memory, fast web, deep research, or another registered tool.



Core rule

Ali - not a hard-coded English router - sees the complete request. Agent Framework constrains the loop: a maximum of eight iterations, one tool call at a time, explicit approval for consequential or costly work, bounded tool output, and compaction before the model context is exhausted.

The solving loop

Accuracy can use more than one step without turning every casual question into a web search.



Decision examples

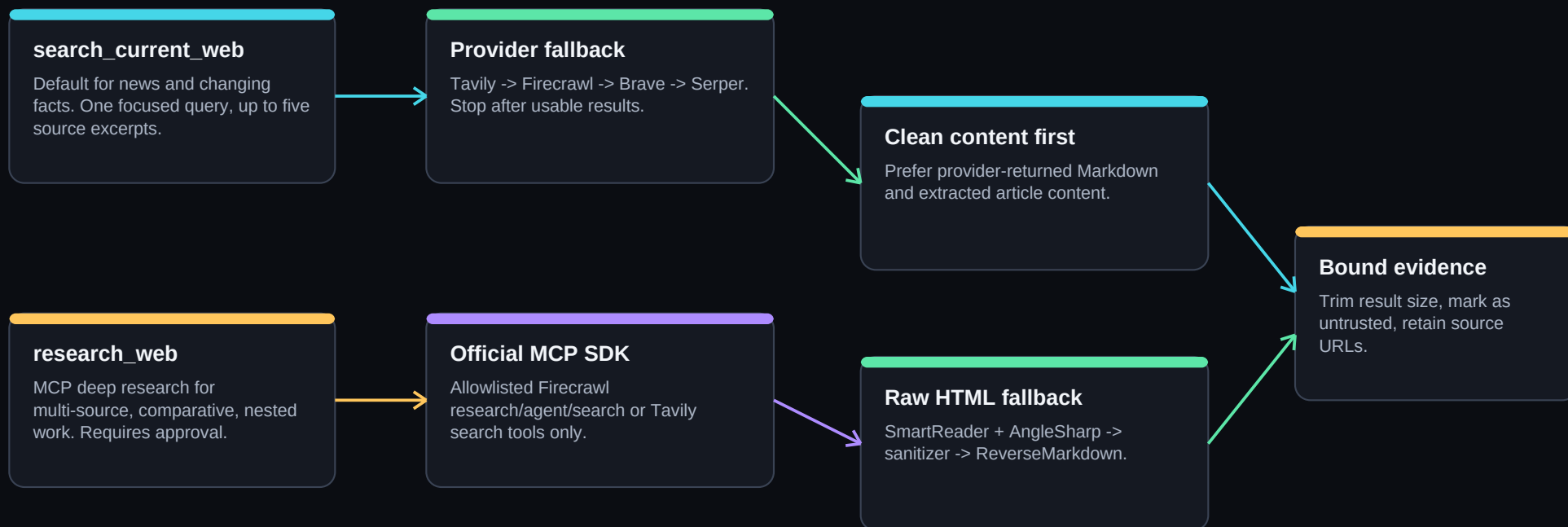
Request	Preferred path	Reason
How are you today?	Direct answer	No sources needed
What was his name?	Memory first	Fast local recall
What happened today?	search_current_web	Focused live evidence
Compare current systems, recalls, cost, warranty	research_web	Nested research + approval
What does my manual say?	search_local_library	Local RAG only

Why this is not hard-coded routing

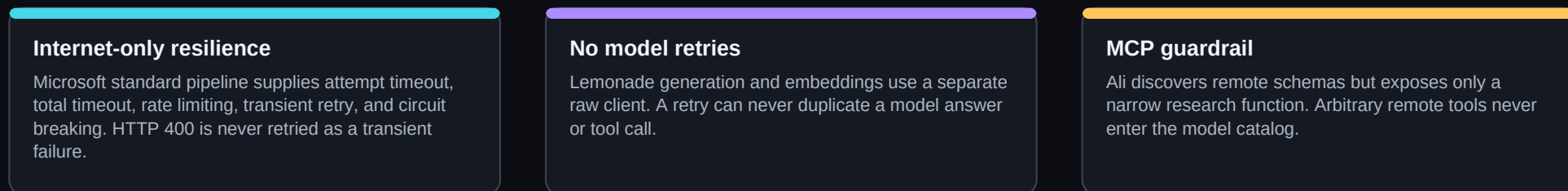
The application defines tool contracts, permissions, limits, and descriptions. GPT-OSS interprets the user's language and chooses among those tools. The code does not attempt to enumerate English phrases or pre-classify every question.

Internet and HTML evidence pipeline

Two speeds: focused current answers stay quick; genuinely nested research gets a stronger provider-managed path.



HTTP reliability boundary



Context, permissions, and failure behavior

The system is designed to fail clearly and preserve control instead of hiding a bad decision.

Context contract

Ali POSTs Lemonade /api/v1/load with the saved ctx_size before generation. Current default: 8,192 context tokens and 2,048 output tokens.

Automatic compaction

Agent Framework receives the same context and output limits, so it compacts sessions before long tool loops exceed Lemonade.

Native tool detection

Lemonade model labels enable native tool calls only for the selected advertised model. Otherwise Ali retains the compatibility client.

Approval policy

Memory writes, reminders, and costly deep research are wrapped with Agent Framework approval. Read-only search and recall run directly.

400 diagnostics

context_length_exceeded becomes a specific explanation with active context and output limits. The request is not blindly retried.

Untrusted evidence

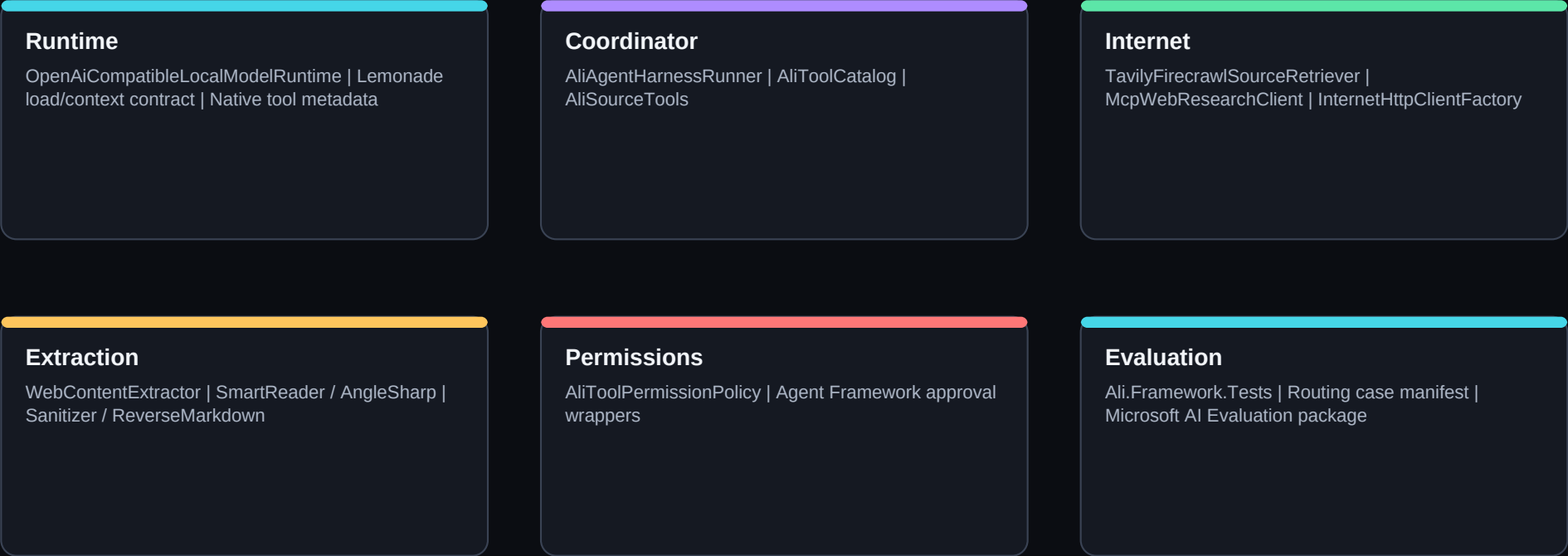
Web pages, MCP results, local documents, and memories are data, never instructions. Result sizes are bounded before re-entering the model.

Visible activity without exposing hidden reasoning

Ali Activity can show: memory checked, tool selected, approval requested, provider queried, source count, fallback used, response completed, and elapsed time. It must not show, speak, save, or reinsert the model's hidden reasoning_content. This gives the user operational control without corrupting future conversation history.

Implementation map and verification

The upgrade stays modular: coordinator policy, runtime transport, internet providers, extraction, permissions, and tests remain separate.



Verified in this change

- Full Ali.sln Release build: zero warnings, zero errors
- Regression tests: Lemonade ctx_size load, native tool metadata, HTML safety, evaluation case integrity
- HTML output is sanitized, converted to Markdown, and size-bounded
- Model traffic and resilient internet traffic use separate HttpClient paths

Libraries: Microsoft Agent Framework Harness 1.15.0 | ModelContextProtocol 1.4.1 | Microsoft.Extensions.Http.Resilience 10.8.0 | AngleSharp 1.5.2 | SmartReader 0.11.1 | ReverseMarkdown 6.1.0 | Meziatou.Framework.HtmlSanitizer 3.0.0 | Microsoft.Extensions.AI.Evaluation.Quality 10.8.0